Cryptographie R309
Outils pour RSA

12 octobre 2026

Objectif Le but de cette séance est de programmer les briques arithmétiques du chiffrement RSA, vues en TD la semaine dernière : PGCD, coefficients de Bézout et inverse modulaire, exponentiation modulaire rapide, tests de primalité. Ces fonctions seront réutilisées telles quelles la semaine prochaine pour construire des clés et chiffrer.

Rendu `rsa_tools.py`

Le code devra être réalisé sous `Python`. Les fonctions demandées (en rose dans l'énoncé) devront être sauvegardées dans un fichier nommé `rsa_tools.py`, à déposer sur le Moodle dans la boîte prévue. Le fichier ne doit contenir ni variable globale ni test : uniquement les fonctions demandées. Les noms de fonctions et l'ordre des arguments doivent être respectés.

Test `test-rsa_tools.py`

Un script de test est fourni afin de valider votre code au fur et à mesure du TP.

Attention : Python dispose de `math.gcd` et de `pow(M, d, n)` qui font exactement ce que l'on demande ici. On vous demande de les *réécrire*, pour en comprendre le fonctionnement ; le script de test vérifie que `pow` à trois arguments n'est pas utilisé.

Exercice 1 PGCD et équation diophantienne

1. *(PGCD)* La génération des clés RSA nécessite de tester si deux entiers sont premiers entre eux, c'est-à-dire si leur plus grand commun diviseur vaut 1. Le calcul du PGCD s'effectue par récurrence (algorithme d'Euclide, vu en TD). Pour $u \geq v > 0$, on effectue la division euclidienne de u par v, de quotient s et de reste r :

$$u = vs + r, \qquad 0 \leq r < v,$$

et l'on a $\mathrm{PGCD}(u, v) = \mathrm{PGCD}(v, r)$. On réitère jusqu'à obtenir un reste nul, et alors $\mathrm{PGCD}(v, 0) = v$.

Algorithme 1 : Calcul du PGCD de u et v

Entrée : u et v
Sortie : $\mathrm{PGCD}(u, v)$

```
si u < v alors
    échanger u et v
si v = 0 alors
    renvoie u
sinon
    soit s le quotient et r le reste de la division euclidienne de u par v;
    renvoie PGCD(v, r)
```

Implémenter une fonction `PGCD` qui prend deux entiers naturels et renvoie leur PGCD.

2. *(Inverse modulaire)* La construction des clés nécessite aussi le calcul d'un inverse modulaire : pour u et v premiers entre eux, trouver a tel que

$$au \equiv 1 \ [v].$$

Implémenter une fonction `inv_mod` qui prend deux entiers u et v et renvoie l'inverse de u modulo v s'il existe, et `None` sinon. On procédera par force brute : tester tous les candidats $a \in \{1, \ldots, v-1\}$.

3. *(Équation diophantienne)* Chercher l'inverse de u modulo v revient à trouver des entiers a et b tels que

$$au + bv = 1,$$

équation diophantienne dont l'existence d'une solution est garantie par le lemme de Bézout.

Lemme 1 (Bézout)**.** *Pour tous entiers u et v, il existe deux entiers a et b tels que*

$$au + bv = \mathrm{PGCD}(u, v).$$

L'algorithme est celui du TD : identique au calcul du PGCD, mais on garde la trace des quotients pour reconstituer a et b en remontant. Sous l'hypothèse $u \geq v > 0$, avec $u = sv + r$, si (A, B) résout $Av + Br = \mathrm{PGCD}(v, r)$, alors

$$\mathrm{PGCD}(u, v) = Av + Br = Av + B(u - sv) = B\,u + (A - Bs)\,v,$$

donc $(a, b) = (B,\ A - Bs)$ convient.

Algorithme 2 : Coefficients de Bézout de u et v

Entrée : u et v
Sortie : (a, b) tel que $au + bv = \mathrm{PGCD}(u, v)$

si $u < v$ **alors**
  $(a, b) \leftarrow$ `Bezout`(v, u);
  renvoie (b, a)
si $v = 0$ **alors**
  renvoie $(1, 0)$
sinon
  soit s le quotient et r le reste de la division euclidienne de u par v;
  $(A, B) \leftarrow$ `Bezout`(v, r);
  renvoie $(B,\ A - Bs)$

Implémenter la fonction `Bezout` correspondante, qui renvoie le couple (a, b).

4. *(Inverse modulaire rapide)* Utiliser `Bezout` pour écrire `inv_mod_fast`, qui prend deux entiers u et v et renvoie l'inverse de u modulo v, réduit dans $\{0, \ldots, v-1\}$, ou `None` si u et v ne sont pas premiers entre eux. Comparer son temps d'exécution à celui de `inv_mod` pour v de l'ordre de 10^8, puis de 10^{30}.

Exercice 2 Puissance modulaire et tests de primalité

1. *(Puissance modulaire)* Chiffrement et déchiffrement RSA reposent sur le calcul de

$$M^d \bmod n$$

où M, d et n sont de grands entiers. Effectuer d multiplications est hors de question : pour d de 1024 bits, l'Univers n'y suffirait pas. On procède par récurrence en divisant l'exposant par deux (exponentiation rapide, vue en TD).

Algorithme 3 : Puissance modulaire $M^d \bmod n$

Entrée : M, d et n
Sortie : $M^d \bmod n$

```
si d = 0 alors
    renvoie 1
sinon
    si d = 1 alors
        renvoie M mod n
    sinon
        soit a le quotient et r le reste de la division de d par 2, de sorte
          que d = 2a + r;
        h ← M^a mod n (appel récursif);
        renvoie h^2 × M^r mod n
```

Implémenter `puissancemodulaire`, qui prend les entiers M, d et n et renvoie $M^d \bmod n$.

Attention : Un seul appel récursif par niveau : calculer M^a deux fois (pour $h \times h$) rendrait l'algorithme aussi lent que la méthode naïve. Combien d'appels récursifs en tout pour d de 1024 bits ?

La génération des clés RSA nécessite de fabriquer de grands nombres premiers. Tester tous les diviseurs possibles est inenvisageable pour des nombres de 300 chiffres. On utilise un test *probabiliste* fondé sur le petit théorème de Fermat.

Théorème 1 (Petit théorème de Fermat)**.** *Soit p un nombre premier et a un entier tel que $1 \leq a < p$. Alors*

$$a^{p-1} \equiv 1 \ [p]. \tag{1}$$

Ainsi, si p ne vérifie pas (1) pour une certaine base a, il n'est pas premier. La réciproque est fausse : il existe des nombres composés qui vérifient (1) pour certaines bases (on les appelle *menteurs*), comme $341 = 11 \times 31$ pour $a = 2$. Mais si p vérifie (1) pour une vingtaine de bases tirées au hasard, la probabilité qu'il soit composé est, sauf cas pathologiques (nombres de Carmichael), extrêmement faible.

2. *(Test de Fermat)* Écrire une fonction `test_fermat` qui prend deux entiers a et p et renvoie `True` si (1) est vérifiée, `False` sinon. Utiliser `puissancemodulaire`. Vérifier que 341 passe le test en base 2 mais pas en base 3.

3. *(Primalité déterministe)* Écrire une fonction `primalite` qui prend un entier n et renvoie `True` si et seulement si n est premier, en vérifiant que n n'admet aucun diviseur entre 2 et $\sqrt{n}$ *inclus*. Pourquoi peut-on s'arrêter à $\sqrt{n}$?

4. *(Test probabiliste)* Écrire une fonction `test_primalite` qui prend deux entiers p et N et renvoie `True` si p passe le test de Fermat pour la base 2 ainsi que pour N bases tirées au hasard dans $\{2, \dots, p-1\}$, `False` dès qu'une base échoue. Comparer la liste des entiers inférieurs à 1000 déclarés premiers par `primalite` et par `test_primalite` avec $N = 5$.

Quelques méthodes utiles en Python. `divmod(u, v)` renvoie le couple quotient, reste ; `random.randint(a, b)` ; `math.isqrt(n)` (racine carrée entière) ; `time.perf_counter()` pour chronométrer.